

Project Type: Graduation Project

Document Version: 5

Date: June 2026

Table of Contents

1.

Project Overview

2.

Project Objectives

3.

Problem Statement

4.

Scope

–

4.1 In-Scope

–

4.2 Out-of-Scope

5.

Team Roles and Responsibilities

6.

Project Timeline

7.

Risk Analysis and Mitigation Plan

8.

Deliverables

9.

Success Criteria

10.

Technical Stack

11.

Conclusion

1. Project Overview

Project Name: Podcast Platform

Codename: SonicAI

Category: Web Application / AI-Enhanced Media Platform

The Podcast Platform is a modern, responsive web application designed to revolutionize how users discover, consume, and interact with podcast content. Built on a React + Vite frontend architecture and powered by the iTunes/Apple Podcasts API, the platform provides seamless access to thousands of podcasts across diverse genres including Technology, Business, Comedy, Health, Education, True Crime, and Sports.

The platform distinguishes itself through the integration of **Google Gemini AI (2.5-flash)**, which enables intelligent content summarization, key insight extraction, and personalized recommendations. Users can browse curated collections, search by topic or creator, stream episodes via an embedded custom audio player, manage personal favorites (per-user persistence), and toggle between light and dark themes.

The system implements a client-side authentication layer using localStorage for session persistence, ensuring users retain their preferences, favorites, and theme settings across sessions. All AI features are gated behind a robust error-handling layer to gracefully manage API quota limits and network failures.

2. Project Objectives

Primary Objectives

ID	Objective	Priority
O1	Develop a responsive, single-page web application for podcast discovery and streaming	High
O2	Integrate iTunes Search API for real-time podcast and episode data retrieval	High
O3	Implement AI-powered content summarization using Google Gemini API	High
O4	Build a custom audio player with playback controls and episode management	High
O5	Design and implement a per-user favorites system with localStorage persistence	Medium
O6	Create a dark/light theme system with system-preference detection	Medium
O7	Implement client-side user authentication (register/login/logout)	Medium

ID	Objective	Priority
O8	Develop comprehensive search functionality with result caching	Medium
O9	Ensure cross-browser compatibility and mobile responsiveness	High
O10	Produce complete technical documentation and user manual	Medium

3. Problem Statement

3.1 Current Landscape

The podcast industry has experienced exponential growth, with over 5 million active podcasts and 70 million+ episodes globally. However, existing platforms suffer from several critical limitations:

- Discovery Overload:** Users are overwhelmed by the sheer volume of content without intelligent filtering or curation mechanisms.
- Time Investment:** Long-form episodes (60–180 minutes) require significant time commitment, yet most platforms offer no concise summary or key-takeaway extraction.
- Fragmented Experience:** Users must switch between multiple apps for discovery, playback, note-taking, and sharing.
- No AI Integration:** Major platforms lack AI-driven summarization, semantic search, or intelligent recommendation engines.
- Poor Personalization:** Favorite lists and preferences are often device-bound or require paid subscriptions.

3.2 Proposed Solution

The Podcast Platform addresses these gaps by providing:

- Unified Discovery & Playback:** A single interface for browsing, searching, and streaming.
- AI Summarization:** One-click generation of structured summaries (Overview, Key Points, Target Audience) for any podcast or episode.
- Smart Curation:** Curated collections, trending algorithms, and genre-based browsing.
- Persistent Personalization:** Per-user favorites and theme preferences stored locally with zero backend dependency for core features.
- Responsive Design:** Full functionality across desktop, tablet, and mobile viewports.

4. Scope

4.1 In-Scope

Functional Requirements

Module	Feature	Description
Discovery	Home Dashboard	Hero search, featured podcasts, popular categories, trending section, stats bar
Discovery	Categories Browsing	Grid of genre cards with iconography and color coding
Discovery	Trending Page	Sort podcasts by popularity metrics with trophy badges
Discovery	Search	Full-text search with results page, query persistence, and loading states
Content	Podcast Details	Cover art, metadata, publisher info, episode list, AI summary panel
Content	Episode Details	Individual episode view with description, audio player, and deep linking
AI	Podcast Summarization	Generate markdown-formatted summaries via Gemini API
AI	Episode Summarization	Per-episode AI analysis with error handling and loading states
Media	Custom Audio Player	Play/pause, progress tracking, minimal mode, episode metadata display
User	Registration	Name, email, phone, password with localStorage persistence
User	Login / Logout	Credential validation, session management, auto-login on app load
User	Favorites	Add/remove podcasts, per-user localStorage, toast notifications

Module	Feature	Description
User	Theme System	Dark/light mode toggle, system preference detection, per-user persistence
UI/UX	Responsive Layout	Sidebar navigation, mobile menu, grid layouts, CSS Modules
UI/UX	Loading States	Spinners, skeleton screens, and progress indicators
UI/UX	Error Handling	404 pages, API failure messages, image fallbacks, quota limit alerts

Non-Functional Requirements

- **Performance:** Initial page load < 3s on 4G networks
- **Compatibility:** Chrome, Firefox, Safari, Edge (last 2 versions)
- **Responsive:** Breakpoints at 576px, 768px, 992px, 1200px
- **Security:** No plaintext password storage (hashed in localStorage), input sanitization
- **Maintainability:** Component-based architecture, CSS Modules, reusable hooks

4.2 Out-of-Scope

Feature	Reason	Future Phase
Backend server / REST API	Client-side only architecture; no Node.js/Express backend	Phase 2
Database (PostgreSQL/MongoDB)	localStorage used for persistence	Phase 2
Social sharing (Twitter/Facebook)	API keys and OAuth complexity	Phase 2
Podcast upload / creator portal	Requires backend and file storage	Phase 3
Offline download / PWA	Service worker implementation deferred	Phase 2
Push notifications	Requires backend and notification service	Phase 3
Multi-language support (i18n)	English-only for graduation scope	Phase 2
Payment/subscription system	No monetization model in academic project	Phase 3
Real-time chat / comments	WebSocket infrastructure not in scope	Phase 3

6.Team Roles and Responsibilities

Adel El-Kady

Team Leader | Front-End Developer | Documentation Specialist

Led the project planning and coordination process, supervised development activities, and contributed to the enhancement of the overall platform. Implemented additional features including the User Analytics Dashboard, listening history tracking, user activity analysis, and personalized insights. Also prepared the project documentation, user manual, technical documentation, testing documentation, and GitHub repository organization.

Habiba mohamed

Project Structure & Organization Coordinator

Designed and organized the project structure, including folder hierarchy and file organization. Contributed to maintaining a clean and scalable project architecture that supports future development and team collaboration.

Jessy kamal

API Integration Specialist

Responsible for researching, evaluating, and integrating external APIs required by the platform. Contributed to data retrieval and content display functionality, including podcast and bootcamp-related content integration within the application.

Mahmoud mohamed

Front-End Developer

Participated in developing and implementing reusable UI components. Contributed to building responsive user interface elements and integrating them within the overall application workflow.

Ibrahim ayman

Front-End Developer

Participated in the development of core application components and user interface implementation. Assisted in building responsive pages and ensuring consistency across different sections of the platform.

6. Project Timeline

6.1 Development Phases

Phase	Duration	Focus
Phase 1: Foundation	Weeks 1–2	Environment setup, architecture, design system
Phase 2: Core Features	Weeks 3–5	API integration, pages, auth, audio player
Phase 3: AI & Polish	Weeks 6–7	Gemini integration, favorites, theme system
Phase 4: Testing & QA	Weeks 8–9	Bug fixes, optimization, cross-browser testing
Phase 5: Documentation & Delivery	Weeks 10–12	Final docs, presentation, deployment

6.3 Milestones

Milestone	Target Week	Deliverable
M1: Architecture Complete	Week 2	Project scaffold, routing, design tokens
M2: Core UI Complete	Week 4	All pages rendered with mock data
M3: API Integration Complete	Week 5	Real data flowing, search working
M4: AI Features Live	Week 7	Summarization functional, favorites stable
M5: Feature Freeze	Week 8	No new features; bug fixes only
M6: Code Freeze	Week 9	All tests passing, performance benchmarked
M7: Documentation Complete	Week 11	All docs reviewed and finalized
M8: Final Presentation	Week 12	Demo delivered, project submitted

7. Risk Analysis and Mitigation Plan

7.1 Risk Register

ID	Risk	Probability	Impact	Mitigation Strategy	
R1	iTunes API rate limiting or deprecation	Medium	High	Implement response caching; design fallback to mock data; monitor API status	
R2	Gemini API quota exhaustion	High	Medium	Implement client-side rate limiting; graceful error messages; fallback to “summary unavailable” UI	
R3	Team member unavailable / sick	Medium	Medium	Cross-train team members; maintain detailed documentation; pair programming sessions	
R4	Scope creep (adding too many features)	High	High	Strict adherence to out-of-scope list; weekly scope review meetings; change request process	
R6	Slow initial load due to large images	Medium	Medium	Implement lazy loading; use placeholder images; optimize image URLs from API	
R7	Security: localStorage credential exposure	Low	High	3	Hash passwords before storage; add input validation; warn users this is a demo system
R8	Responsive	Medium	Medium	4	Mobile-first CSS approach; test on physical devices; Designer

ID	Risk	Probability	Impact	Mitigation Strategy		
R10	design breaks on small devices	High	High	9	use Bootstrap grid + custom breakpoints	
	Academic deadline pressure				Buffer time in schedule (Week 11–12); prioritize MVP features; daily stand-up meetings	Project Manager

7.2 Contingency Plans

- **If iTunes API fails entirely:** Switch to fully mock-data mode with pre-loaded podcast JSON.
- **If Gemini API is unavailable:** Disable AI features; display “AI features temporarily unavailable” banner.
- **If major bug found in Week 9:** Use Week 10 buffer for hotfixes; defer non-critical documentation.

9. Success Criteria

9.1 Functional Success Criteria

Criterion	Measurement	Target
SC1	All primary features implemented	100% of O1–O10 complete
SC2	Search returns results in < 2 seconds	Average of 10 queries
SC3	AI summary generation time	< 5 seconds per request
SC4	Audio player plays without errors	0 critical playback bugs
SC5	Favorites persist across sessions	100% retention after refresh
SC6	Auth system validates credentials	0 false positives/negatives

10. Technical Stack

10.1 Frontend

Technology	Version	Purpose
React	18.x	UI library
Vite	5.x	Build tool & dev server
React Router DOM	6.x	Client-side routing
CSS Modules	Native	Scoped component styling
Font Awesome	6.x	Iconography
React Hot Toast	2.x	Notification system
React Markdown	9.x	AI summary rendering

10.2 APIs & Services

Service	Purpose
iTunes Search API	Podcast metadata, episodes, search
Google Gemini 2.5-flash	AI summarization & content analysis

10.4 Environment Variables

VITE_GEMINI_API_KEY=your_gemini_api_key_here

11. Conclusion

The Podcast Platform (SonicAI) represents a comprehensive graduation project that combines modern frontend engineering with practical AI integration. By leveraging the iTunes API for content and Google Gemini for intelligent analysis, the platform delivers genuine value to users overwhelmed by the modern podcast landscape.

The project planning document establishes clear objectives, realistic timelines, defined roles, and robust risk mitigation strategies. With disciplined adherence to the week-by-week schedule and proactive scope management, the team is positioned to deliver a high-quality, feature-rich application that meets both academic standards and real-world usability expectations.

The modular architecture (React + CSS Modules + Context API) ensures maintainability and extensibility, allowing future phases to introduce backend services, user-generated content, and advanced recommendation algorithms without architectural overhaul.

Document Prepared By: [Adel abd elzaher]

Supervised By: [Dr. Mahmoud raslan]

Front end react track

DEPI

June 2026
